

KUBERNETES

VOLÚMENES

KUBERNETES

ALMACENAMIENTO

Ya lo hemos comentando en anteriores módulos, pero es importante tener en cuenta que **los Pods son efímeros**, es decir, cuando un Pod se elimina se pierde toda la información que tenía. Evidentemente, cuando creamos un nuevo Pod no contendrá ninguna información adicional a la propia aplicación.

La solución consiste en usar Volúmenes. Con el uso de dichos volúmenes vamos a conseguir varias cosas:

1. Si un Pod guarda su información en un volumen, está no se perderá. Por lo que podemos eliminar el Pod sin ningún problema y cuando volvamos a crearlo mantendrá la misma información. En definitiva, los volúmenes proporcionan almacenamiento adicional o secundario al disco que define la imagen.
2. Si usamos volúmenes, y tenemos varios Pods que están ofreciendo un servicio, estos Pods tendrán la información compartida y por tanto todos podrán leer y escribir la misma información.
3. También podemos usar los volúmenes dentro de un Pod, para que los contenedores que forman parte de él puedan compartir información.

KUBERNETES

ALMACENAMIENTO

Tipos de volúmenes

- Proporcionados por proveedores de cloud: AWS, Azure, GCE, OpenStack, etc
- Propios de Kubernetes: **kubectl get storageClass**
 - configMap: Para usar un configMap como un directorio desde el Pod.
 - emptyDir: Volumen efímero con la misma vida que el Pod. Usado como almacenamiento secundario o para compartir entre contenedores del mismo Pod.
 - hostPath: Monta un directorio del host en el Pod (usado excepcionalmente, pero es el que nosotros vamos a usar con minikube).
 - ...
- Habituales en despliegues "on premises": glusterfs, cephfs, iscsi, nfs, etc.

KUBERNETES

ALMACENAMIENTO

Aprovisionamiento estático PersistentVolumen (PV).

Tenemos tres modos de acceso, que dependen del backend que vamos a utilizar:

- ReadWriteOnce: read-write solo para un nodo (RWO)
- ReadOnlyMany: read-only para muchos nodos (ROX)
- ReadWriteMany: read-write para muchos nodos (RWX)

Las políticas de reciclaje de volúmenes también dependen del backend y son:

- Retain: El PV no se elimina, aunque el PVC se elimine. El administrador debe borrar el contenido para la próxima asociación.
- Recycle: Reutilizar contenido. Se elimina el contenido y el volumen es de nuevo utilizable. **OBSOLETO**
- Delete: Se borra después de su utilización.

A modo de resumen, ponemos en la siguiente tabla los modos de acceso de algunos de los sistemas de almacenamiento más usados:

KUBERNETES

ALMACENAMIENTO

Plugin	ReadWriteOnce	ReadOnlyMany	ReadWriteMany
AWS EBS	✓	-	-
AzureFile	✓	✓	✓
AzureDisk	✓	-	-
CephFS	✓	✓	✓
Cinder	✓	-	-
GCEPersistentDisk	✓	✓	-
Glusterfs	✓	✓	✓
HostPath	✓	-	-
iSCSI	✓	✓	-
NFS	✓	✓	✓
RBD	✓	✓	-

KUBERNETES

Aprovisionamiento manual persistente: ADMINISTRADOR

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv1
spec:
  storageClassName: manual
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  hostPath:
    path: /data/pv1
```

KUBERNETES

ALMACENAMIENTO

PersistentVolumen (PVC).

Cuando el desarrollador necesita almacenamiento para su aplicación, hace una petición de almacenamiento creando un recurso PersistentVolumeClaim (PVC) y de forma dinámica se crea el recurso PersistentVolume que representa el volumen y se asocia con esa petición. De otra forma explicado, cada vez que se cree un PersistentVolumeClaim, se creará bajo demanda un PersistentVolumen que se ajuste a las características seleccionadas.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvcl
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

KUBERNETES

Una vez que hemos solicitado el almacenamiento, y se ha asignado un volumen (ya sea de forma dinámica o estática), vamos a definir el uso que se va a hacer de este volumen.

Como ejemplo, vamos a definir un Pod que utilice dicho volumen, para ello vamos a crear un fichero yaml con la siguiente definición:

```
kind: Pod
apiVersion: v1
metadata:
  name: task-pv-pod
spec:
  volumes:
    - name: task-pv-storage
      persistentVolumeClaim:
        claimName: pvcl
  containers:
    - name: task-pv-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: task-pv-storage
```

- Para probarlo una vez creado todo, cambiamos la página:
kubectll exec pod/nginx-ejemplo1-86864d84b5-s62dq -- bash -c "echo '<h1>Almacenamiento en K8S</h1>' > /usr/share/nginx/html/index.html"
- Accedemos através del service
- Borramos el deployment
- Ya no se puede acceder
- Crearmos uno nuevo y tiene que mostrar la misma página.